



**Evaluación conjunta: Auditoría de Seguridad y Despliegue en Kubernetes del
Sistema InvenTrack**

Marcos Escobar

Universidad de las Fuerzas Armadas ESPE

30735 — Desarrollo de Software Seguro

Ing. Geovanny Cudco

31 de julio de 2026



Evaluación conjunta: Auditoría de Seguridad y Despliegue en Kubernetes del

Sistema InvenTrack

Índice

Introducción	5
Objetivo General	5
Objetivos Específicos	5
Enlace al Repositorio de Manifiestos de Kubernetes	6
Dockerización de Componentes	6
Backend (Node.js/Express)	6
Frontend (React/Vite)	7
Base de Datos (MySQL)	8
Evidencia: Contenedores en Ejecución	8
Manifiestos de Kubernetes	9
Namespaces	9
Deployments y Services	9
ConfigMaps y Secrets	9
PersistentVolumeClaim (PVC)	9
Ingress	9
Despliegue en el Clúster y Configuración de Ingress	9
Descripción del Clúster	9
Controlador Ingress	9
Resolución del Dominio conjunta3p.espe.edu.ec	9

Evidencia: Recursos en Kubernetes	10
Evidencia: Aplicación en el Navegador	10
Auditoría de Seguridad: Pentesting con Metodología PTES	10
Fase 1: Interacciones Preliminares (Pre-engagement) — 5 %	10
Alcance del Pentesting	10
Objetivos y Reglas de Juego	10
Activos a Probar	10
Autorización	10
Fase 2: Recopilación de Inteligencia (Intelligence Gathering) — 8 %	10
Mapeo de Puertos (Nmap)	10
Detección de Tecnologías (WhatWeb / Wappalyzer)	11
Cabeceras HTTP y Rutas de Ingress	11
Fase 3: Modelado de Amenazas (Threat Modeling) — 7 %	11
Fase 4: Análisis de Vulnerabilidades (Vulnerability Analysis) — 12 %	11
Escaneo con Nikto	11
Escaneo con OWASP ZAP / Burp Suite	12
Análisis de Cabeceras de Seguridad	12
Resumen de Vulnerabilidades Detectadas	12
Fase 5: Explotación (Exploitation) — 15 %	12
Intento de Bypass de Autenticación	12
Fuerza Bruta contra el Login (Hydra / Burp Intruder)	12
Inyección SQL (Login / Chatbot)	13
Pruebas de IDOR (Endpoints de Auditoría / Kardex)	13
Fase 6: Post-Explotación (Post-Exploitation) — 8 %	13
Análisis de Impacto	13
Validación del Hasheo con bcrpt	13
Escalación de Privilegios (admin vs almacenero)	13
Fase 7: Reporte (Reporting) — 5 %	13

Clasificación de Hallazgos por Severidad	13
Validación de Controles de Seguridad	13
Conclusiones y Recomendaciones	14
Conclusiones	14
Recomendaciones de Mejora para InvenTrack	14

Introducción

El presente informe detalla el proceso de containerización, despliegue en un clúster de Kubernetes y auditoría de seguridad del sistema InvenTrack. Se documentan los pasos realizados para la creación de contenedores Docker para los componentes del sistema (backend, frontend y base de datos), la configuración de manifiestos de Kubernetes, el despliegue en un clúster local, y la configuración del Ingress para exponer la aplicación bajo el dominio `conjunta3p.espe.edu.ec`. Además, se presenta un análisis de seguridad siguiendo la metodología PTES (Penetration Testing Execution Standard), incluyendo la recopilación de inteligencia, modelado de amenazas, análisis de vulnerabilidades, explotación y post-explotación, así como las conclusiones y recomendaciones derivadas del proceso.

Objetivo General

El objetivo general de esta evaluación es realizar un despliegue seguro del sistema InvenTrack en un clúster de Kubernetes, asegurando la correcta containerización de sus componentes y evaluando su seguridad mediante pruebas de penetración siguiendo la metodología PTES.

Objetivos Específicos

- Realizar la containerización de los componentes del sistema InvenTrack (backend, frontend y base de datos) utilizando Docker.
- Configurar los manifiestos de Kubernetes necesarios para ejecutar InvenTrack en un clúster, incluyendo Deployments, Services, ConfigMaps, Secrets, PersistentVolumeClaims e Ingress.
- Configurar el Ingress para exponer la aplicación bajo el dominio `conjunta3p.espe.edu.ec` y asegurar la correcta resolución de DNS.
- Realizar una auditoría de seguridad del sistema InvenTrack siguiendo la metodología PTES, identificando vulnerabilidades y evaluando la efectividad de los controles de seguridad implementados.

Enlace al Repositorio de Manifiestos de Kubernetes

Repositorio GitHub (manifiestos .yaml):

https://github.com/IMarcusDev/Evaluacion_Final_k8s_30735_DSS/tree/main

Dockerización de Componentes

La containerización del sistema InvenTrack se organizó en tres componentes independientes: el backend (API REST en Node.js/Express), el frontend (aplicación React construida con Vite) y la base de datos MySQL. Para cada componente se definió una estrategia de empaquetado acorde con su función. El backend y el frontend se construyeron a partir de un Dockerfile propio, mientras que la base de datos se apoyó en la imagen oficial de MySQL 8.0, configurada por variables de entorno y volúmenes. En todos los casos se partió de imágenes base Alpine, que reducen el tamaño de la imagen resultante. A continuación se describen los Dockerfiles de cada componente y las decisiones de construcción aplicadas.

Backend (Node.js/Express)

```
1  #Marcos Escobar
2  FROM node:20-alpine
3
4  WORKDIR /app
5
6  COPY package*.json ./
7  RUN npm ci --omit=dev
8
9  COPY . .
10
11 RUN mkdir -p uploads \
12     && addgroup -g 1001 -S app \
13     && adduser -u 1001 -S app -G app \
14     && chown -R app:app /app
15 USER app
16
17 EXPOSE 4000
18
19 CMD ["node", "src/app.js"]
```

Listing 1: Dockerfile del Backend

El Dockerfile del backend, presentado en el Listado 1, partió de la imagen `node:20-alpine`. Primero se copiaron los archivos `package*.json` y se ejecutó `npm ci --omit=dev`, de modo que la imagen incorpora solo las dependencias de producción (`--omit=dev` es la versión moderna de `--production`). Después se copió el código fuente y se creó el directorio `uploads`, destinado a las imágenes de producto que gestiona Multer. Un aspecto relevante para la seguridad fue la creación de un usuario sin privilegios (`app`, con UID y GID 1001) y la asignación de la propiedad del directorio de trabajo a dicho usuario. La instrucción `USER app` evitó que el proceso se ejecute como `root` dentro del contenedor, lo que reduce el impacto de una eventual intrusión. Por último se expuso el puerto 4000 y se definió el arranque del servidor con `node src/app.js`.

Frontend (React/Vite)

```
1 #Marcos Escobar
2 FROM node:20-alpine AS build
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci
6 COPY . .
7 RUN npm run build
8
9 FROM nginx:1.27-alpine
10 COPY --from=build /app/dist /usr/share/nginx/html
11 COPY nginx.conf /etc/nginx/conf.d/default.conf
12 EXPOSE 80
```

Listing 2: Dockerfile del Frontend (multi-stage)

El Dockerfile del frontend, mostrado en el Listado 2, aplicó una construcción en dos partes (multi-stage). En la primera parte, basada en `node:20-alpine`, se instalaron las dependencias con `npm ci` y se generó la versión estática de producción con `npm run build`, que Vite genera en el directorio `dist`. En la segunda etapa se utilizó la imagen `nginx:1.27-alpine` como servidor

web. Solo los archivos compilados del directorio `dist` se copiaron a la ruta pública de Nginx, junto con una configuración personalizada (`nginx.conf`) que define el enrutamiento de la aplicación de página única y agrega cabeceras de seguridad. De este modo, la imagen final no contiene el código fuente ni las herramientas de compilación, lo que disminuye su tamaño y su superficie de exposición. El contenedor del frontend expone el puerto 80.

Base de Datos (MySQL)

La base de datos se ejecutó a partir de la imagen oficial `mysql:8.0`, sin un Dockerfile propio, dado que la configuración requerida se resolvió por variables de entorno y volúmenes. La persistencia de la información se garantizó con un volumen montado en `/var/lib/mysql`, de modo que los datos sobreviven al reinicio o a la recreación del contenedor. El esquema inicial se cargó de forma automática: los archivos de inicialización se montaron en el directorio `/docker-entrypoint-initdb.d`, que la imagen de MySQL ejecuta en la primera puesta en marcha cuando el directorio de datos está vacío. Ese conjunto incluyó el archivo `init.sql`, con la definición de las tablas, y un archivo de datos de ejemplo (`seed_01_electronica.sql`) que inserta usuarios, categorías, proveedores, productos y movimientos de kardex. Con este mecanismo, el sistema dispuso de datos iniciales sin una carga manual posterior. La orquestación de los tres contenedores se realizó con Docker Compose, que además definió un *healthcheck* sobre MySQL para que el backend inicie solo cuando la base de datos está operativa.

Evidencia: Contenedores en Ejecución

La construcción y el arranque de los tres contenedores se validaron con el comando `docker compose up --build`. La Figura 1 presenta la salida de `docker ps` tras el despliegue local. En ella se observa que los servicios `inventrack-frontend`, `inventrack-backend` e `inventrack-mysql` se encuentran en estado `Up`, con el contenedor de la base de datos marcado como `healthy`. Los puertos publicados confirman la exposición del frontend en el 5173, del backend en el 4000 y de MySQL en el 3306.


```

kernel      7.1.4-arch1-1
uptime      1 hour, 21 minutes
shell       fish
mem         11.44 GiB / 14.88 GiB
pkgs        1799
user        imarcus
hname       IMarcusDev
distro      Arch Linux

➜ 0s ○ docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
NAMES          IMAGE                                STATUS          PORTS
inventrack-frontend  conjunta-desarrollo-seguro-frontend Up 3 minutes    0.0.0.0:5173->80/tcp, [::]:5173->80/tcp
inventrack-backend  conjunta-desarrollo-seguro-backend Up 3 minutes    0.0.0.0:4000->4000/tcp, [::]:4000->4000/tcp
inventrack-mysql    mysql:8.0                        Up 3 minutes (healthy) 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp

➜ 0s ○ docker ps
CONTAINER ID   IMAGE                                NAMES          COMMAND          CREATED          STATUS          PORTS
ef4afbee6455   conjunta-desarrollo-seguro-frontend inventrack-frontend "/docker-entrypoint..." 54 minutes ago   Up 54 minutes   0.0.0.0:5173->80/t
cp, [::]:5173->80/tcp
9a842d157be0   conjunta-desarrollo-seguro-backend inventrack-backend "docker-entrypoint.s..." 54 minutes ago   Up 54 minutes   0.0.0.0:4000->4000
/tcp, [::]:4000->4000/tcp
904463ac13c3   mysql:8.0                        inventrack-mysql "docker-entrypoint.s..." 54 minutes ago   Up 54 minutes (healthy) 0.0.0.0:3306->3306
/tcp, [::]:3306->3306/tcp, 33060/tcp

```

Figura 1

Contenedores en ejecución (*docker ps*).

Manifiestos de Kubernetes

Namespaces

Deployments y Services

ConfigMaps y Secrets

PersistentVolumeClaim (PVC)

Ingress

Despliegue en el Clúster y Configuración de Ingress

Descripción del Clúster

Controlador Ingress

Resolución del Dominio conjunta3p.espe.edu.ec

```
1 # >>> Ejemplo: 192.168.49.2    conjunta3p.espe.edu.ec <<<
```

Listing 3: Configuración de `/etc/hosts` para el entorno de prueba

Evidencia: Recursos en Kubernetes**Figura 2**

Pods, Services e Ingress en ejecución (`kubectl get all`).

Evidencia: Aplicación en el Navegador**Figura 3**

InvenTrack accesible bajo el dominio `conjunta3p.espe.edu.ec`.

Auditoría de Seguridad: Pentesting con Metodología PTES**Fase 1: Interacciones Preliminares (Pre-engagement) — 5 %**

Alcance del Pentesting

Objetivos y Reglas de Juego

Activos a Probar

Autorización

Fase 2: Recopilación de Inteligencia (Intelligence Gathering) — 8 %

Mapeo de Puertos (Nmap)

```
1 # nmap -sV -p- conjunta3p.espe.edu.ec
2 # >>> PEGAR COMANDO Y RESULTADO <<<
```

Listing 4: Comando Nmap ejecutado

Figura 4

Resultado del escaneo de puertos con Nmap.

Detección de Tecnologías (WhatWeb / Wappalyzer)

Cabeceras HTTP y Rutas de Ingress

Fase 3: Modelado de Amenazas (Threat Modeling) — 7 %

Tabla 1

Matriz de modelado de amenazas de InvenTrack

Activo	Vector de Ataque	Superficie de Exposición
API REST		
Autenticación JWT		
Carga de archivos (Multer)		
Carga masiva Excel/CSV		
Chatbot (Groq)		
Base de datos MySQL		

Fase 4: Análisis de Vulnerabilidades (Vulnerability Analysis) — 12 %

Escaneo con Nikto

```
1 # nikto -h https://conjunta3p.espe.edu.ec
2 # >>> PEGAR COMANDO Y RESULTADO <<<
```

Listing 5: Comando Nikto ejecutado

Escaneo con OWASP ZAP / Burp Suite

Análisis de Cabeceras de Seguridad

Resumen de Vulnerabilidades Detectadas

Tabla 2

Vulnerabilidades detectadas en la fase de análisis

ID	Descripción	Severidad (CVSS)
V-01		
V-02		
V-03		

Fase 5: Explotación (Exploitation) — 15 %

Intento de Bypass de Autenticación

Fuerza Bruta contra el Login (Hydra / Burp Intruder)

```
1 # hydra -l admin -P rockyou.txt conjunta3p.espe.edu.ec http-post-form ...
```

Listing 6: Comando de fuerza bruta ejecutado

Inyección SQL (Login / Chatbot)

Pruebas de IDOR (Endpoints de Auditoría / Kardex)

Fase 6: Post-Explotación (Post-Exploitation) — 8 %

Análisis de Impacto

Validación del Hasheo con bcrypt

Escalación de Privilegios (admin vs almacenero)

Fase 7: Reporte (Reporting) — 5 %

Clasificación de Hallazgos por Severidad

Tabla 3

Hallazgos clasificados por severidad CVSS

ID	Hallazgo	CVSS	Severidad
H-01			
H-02			
H-03			

Validación de Controles de Seguridad

Tabla 4

Validación de controles de seguridad de InvenTrack

Control	Prueba Realizada	Resultado
Rate limiting		
Hasheo bcrypt		
Tokens JWT		
Verificación de sesión (5s)		

Conclusiones y Recomendaciones

Conclusiones

Recomendaciones de Mejora para InvenTrack

-
-
-

Apéndice

Apéndice A: Evidencias Adicionales